



привет

нужно определить софт для конфигурации сервера Ubuntu Linux с 2 h 100 и 2 ерус 9554
нужно иметь возможность выполнять код, распараллеливать вычисление нескольких
моделей одновременно и так далее,максимально универсально,опиши максимально
подробно и максимально много софта

Для Ubuntu-сервера с 2× NVIDIA H100 и 2× AMD EPYC 9554 самый универсальный подход — собрать “слоёный” стек: базовая ОС/драйверы → контейнеры → планировщик/оркестрация → фреймворки распределённых вычислений → мониторинг/профилирование. Для параллельного запуска нескольких моделей одновременно ключевые рычаги — **MIG** (аппаратное разбиение GPU), планировщик (Slurm/Kubernetes) и правильная изоляция ресурсов на уровне контейнеров/cgroups. ^{[1] [2]}

База ОС и GPU-стек

Начать стоит с LTS Ubuntu Server (22.04/24.04), актуального NVIDIA driver (data center ветка) и пользовательских библиотек CUDA-экосистемы (CUDA Runtime, cuDNN, TensorRT/пакеты для инференса), чтобы и обучение, и инференс работали на одном узле без “зоопарка” установок.

- MIG (Multi-Instance GPU): позволяет делить один физический GPU на несколько изолированных GPU-инстансов, что удобно для одновременного запуска нескольких моделей/пользователей (например, отдельный MIG-слайс под инференс, другой — под обучение). ^[1]
- NCCL: базовая библиотека коллективных коммуникаций для multi-GPU/multi-node (all-reduce/all-gather и т.п.), оптимизированная под PCIe/NVLink/NVSwitch и сетевые транспорты (IB/TCP), обычно является “движком” для distributed-тренировки. ^[3]
- Режимы конкурентного доступа: MIG даёт жёсткую изоляцию на уровне “кусочков” GPU, а планировщик затем раздаёт эти ресурсы задачам (по сути, “сколько инстансов/сколько GPU”). ^{[1] [2]}

Контейнеры и окружения

Контейнеризация почти всегда выигрывает по универсальности: разные проекты/версии PyTorch, разные CUDA dependencies, быстрый откат, воспроизводимость, и проще параллелить несколько моделей.

- NVIDIA Container Toolkit: официальный стек, который “подключает” GPU в контейнеры и работает с Docker/containerd/CRI-O/Podman (при установленном драйвере на хосте). ^{[4] [5]}

- Docker / containerd + Compose: удобно для “одна машина = один владелец/команда”, быстрый старт для инференса/батчей (vLLM/Triton + отдельные сервисы).
- Apptainer (ex-Singularity): часто выбирают в HPC/Slurm-средах, когда важны безопасность, неизменяемые образы и запуск без root в compute-окружении (как альтернатива Docker на кластере).

Планировщик и параллелизм

Если надо “распараллеливать вычисление нескольких моделей одновременно” (и особенно если пользователей/задач много), без планировщика обычно быстро наступает конфликт за GPU/CPU/память.

Ниже — самые практичные варианты (можно смешивать, но лучше выбрать “стержень”):

Вариант	Когда выбирать	Что даёт для GPU/параллельности
Slurm	HPC-стиль: очереди, батчи, fair-share, отчётность	Умеет выдавать GPU только по явному запросу (<code>--gres</code>) и управлять GRES (включая <code>AutoDetect=nvml</code>), что удобно для строгого разделения ресурсов между задачами. ^[2]
Kubernetes + NVIDIA GPU Operator	“Cloud-native”: сервисы, автоскейл, GitOps	GPU Operator умеет разворачивать MIG Manager для управления MIG-конфигурацией на нодах кластера. ^[6]
“Лёгкий” режим без оркестрации	Один пользователь/одна команда, мало конкуренции	Разруливание руками через <code>CUDA_VISIBLE_DEVICES</code> , <code>systemd/cgroups</code> , Docker Compose; масштабировать по людям и проектам тяжело.

Дополнительно по параллелизму (по слоям):

- **На уровне GPU:** MIG для одновременных независимых задач на одном H100. ^[1]
- **На уровне задач:** Slurm раздаёт GPU через GRES/`--gres` и не “переподписывает” ресурсы, если вы не разрешите это явно политиками. ^[2]
- **На уровне ML-кода:** PyTorch DistributedDataParallel обычно запускается с процесс-группой и часто использует `backend='nccl'` для GPU-коммуникаций. ^[7]

ML/AI стек (обучение и инференс)

Чтобы стек был максимально универсальным, обычно держат 2 “линии”: (1) обучение/исследования, (2) прод-инференс/сервинг.

Обучение/распределёнка:

- PyTorch (+ DDP/FSDP), Lightning, DeepSpeed, Hugging Face Accelerate/Transformers (быстро включать multi-GPU, mixed precision, ZeRO и т.д.).
- Ray (train/serve), Dask, MPI/OpenMPI/UCX (если много HPC-логики/параллельных пайплайнов).
- JupyterLab/JupyterHub (если нужно многопользовательски), VS Code Server/code-server, tmux/screen (для сессий).

Инференс/сервинг (несколько моделей одновременно):

- NVIDIA Triton Inference Server (универсальный “батчер/сервер” для нескольких моделей и версий, удобно под нагрузку).
- vLLM / TGI (Text Generation Inference) для LLM-сервинга, часто проще, чем “самописный” FastAPI вокруг модели.
- TensorRT / TensorRT-LLM (когда важны оптимизация графа, latency и throughput на NVIDIA GPU).

Мониторинг, профилирование, безопасность

На 2× H100 мониторинг обязателен: иначе сложно понять, почему “всё медленно” и кто съел GPU/VRAM/PCIe.

- DCGM Exporter: экспортирует GPU-метрики в формате, который может собирать Prometheus (типовая связка — Prometheus + Grafana). [\[8\]](#) [\[9\]](#)
- Prometheus + Grafana: метрики/дашборды (GPU util, память, температура, power, ошибки), алерты, SLO. [\[8\]](#)
- Профилирование: NVIDIA Nsight Systems / Nsight Compute (узкие места kernel’ов/ пайплайна), плюс Python-профайлеры (py-spy, scalene) и системные (perf, bpftrace).
- Безопасность/изоляция: SSH hardening, fail2ban, ufw/nftables, auditd, SELinux/AppArmor; плюс строгие политики контейнеров (rootless где возможно) и раздача GPU только через планировщик.

Какой режим эксплуатации планируется: один владелец (личная/командная машина) или много пользователей с очередью задач (мини-кластер внутри одной машины)? Это определит, выбрать Slurm+Apptainer или Kubernetes+GPU Operator.



а можно установить и то и то, а на практике уже сделать выбор?

Да, можно поставить и Slurm, и Kubernetes (с NVIDIA GPU Operator), а потом на практике выбрать — но лучше избегать ситуации, когда **оба** одновременно пытаются “владеть” одними и теми же GPU/ MIG-профилями. [\[31\]](#) [\[32\]](#)

Самый безопасный путь — установить оба стека, но включать в работу по очереди, либо жёстко разделить GPU (или MIG-инстансы) между ними. [\[33\]](#) [\[31\]](#)

Что реально конфликтует

Главный источник конфликтов — управление MIG-геометрией и “кто раздаёт GPU задачам”. [\[32\]](#) [\[31\]](#)

В Kubernetes GPU Operator использует MIG Manager, который применяет MIG-конфигурацию по node label `nvidia.com/mig.config` и при этом останавливает GPU-поды, а при “preinstalled drivers” может также останавливать хостовые

systemd-сервисы из списка GPU-клиентов, чтобы выполнить MIG-ре-конфигурацию. [\[34\]](#)
Если в этот момент Slurm (или любые non-K8s задачи) используют GPU, они могут быть прерваны/сломаны, поэтому одновременная эксплуатация без разделения ресурсов обычно заканчивается болью. [\[34\]](#) [\[31\]](#)

Как поставить оба и “поиграться”

Практичная схема “установить и выбрать позже” выглядит так:

- Общая база: один набор NVIDIA драйверов на хосте + один контейнерный runtime (Docker/containerd) как фундамент для обоих подходов.
- Slurm: поставить пакеты и настроить GRES для GPU (Slurm умеет выделять generic resources под запрос задачи и не отдаёт их другим задачам, пока job их держит). [\[31\]](#)
- Kubernetes: поднять тестовый кластер (часто k3s/ kubeadm) и поставить NVIDIA GPU Operator только когда нужно тестировать K8s-подход с GPU (включая MIG). [\[32\]](#)

Так можно по неделям/по проектам переключаться: “сейчас весь узел = Slurm”, “сейчас весь узел = Kubernetes”, не мешая системам друг другу. [\[31\]](#) [\[32\]](#)

Если хочется одновременно: как разделять GPU

Одновременная эксплуатация возможна, если **жёстко** разделить ресурсы так, чтобы планировщики не пересекались. [\[31\]](#) [\[35\]](#)

Рабочие варианты:

- Разделить по физическим GPU: например, GPU0 отдаётся только Kubernetes, GPU1 — только Slurm (на уровне конфигов и политики запуска).
- Разделить через MIG: заранее нарезать MIG-инстансы и часть “слайсов” считать ресурсами K8s, часть — ресурсами Slurm (требует дисциплины, чтобы K8s-компоненты не меняли MIG-геометрию в рантайме). [\[33\]](#) [\[34\]](#)
- В Kubernetes: настроить NVIDIA k8s-device-plugin с MIG_STRATEGY (есть стратегии none | single | mixed; при mixed появляются ресурсы вида nvidia.com/mig-<slice_count>g.<memory_size>gb, которые можно запрашивать в pod spec). [\[35\]](#)

Критичный момент: если разрешить GPU Operator'у динамически “пере-нарезать” MIG (через label nvidia.com/mig.config), это может затрагивать не только pod'ы, но и хостовые GPU-клиенты, поэтому для совместной жизни чаще фиксируют MIG-геометрию и запрещают её менять автоматически. [\[34\]](#) [\[32\]](#)

Практический критерий выбора (что оставить в проде)

Slurm обычно проще и надёжнее для батч-тренировок/экспериментов “запустил N задач, подожди, заberi результат” и строгой раздачи GPU через GRES. [\[31\]](#)

Kubernetes обычно удобнее для продового инференса и множества сервисов (много деплоймов, rolling updates, сетевые политики), и там же удобно управлять MIG через GPU Operator/MIG Manager. [\[32\]](#)

Сколько пользователей и какой основной профиль нагрузки планируется: больше “батчи/обучение” или “онлайн-инференс/сервисы”?

✱

теперь дай мануал, как жто все устанавливать

Ниже — практический “плейбук” установки универсального стека на Ubuntu для 2×H100 + 2×EPYC: общий слой (драйвер/MIG/контейнеры) + две ветки (Slurm и Kubernetes), чтобы потом переключаться. Часть шагов намеренно оформлена так, чтобы Kubernetes и Slurm не “дрались” за MIG: Slurm предполагает, что MIG уже нарезан и **не** умеет динамически менять MIG-геометрию. [\[47\]](#)

Общая база: драйвер → MIG → контейнеры

Сначала ставится NVIDIA datacenter driver, проверяется `nvidia-smi`, затем включается MIG и создаются GPU/Compute Instances, потому что одного “MIG enabled” недостаточно для запуска CUDA-нагрузок. [\[48\]](#)

На Hopper (H100) включение MIG не требует GPU reset, но состояние MIG не является постоянным после перезагрузки/перезагрузки модулей драйвера — нужно автоматизировать восстановление геометрии на старте. [\[48\]](#)

1. Обновить ОС и базовые пакеты:

```
sudo apt update
sudo apt -y install build-essential dkms linux-headers-$(uname -r) curl gnupg2
sudo reboot
```

2. Установить NVIDIA driver (рекомендуется через пакетный менеджер дистрибутива).

Проверка:

```
nvidia-smi
```

3. Включить MIG (на каждом GPU):

```
sudo nvidia-smi -i 0 -mig 1
sudo nvidia-smi -i 1 -mig 1
nvidia-smi --query-gpu=pci.bus_id,mig.mode.current --format=csv
```

На H100 режим MIG включён, но после ребута геометрию придётся воссоздавать — учитывая это сразу при дизайне (systemd unit / скрипт инициализации). [\[48\]](#)

4. Создать MIG-инстансы (пример; профили подбираются под твои модели):

```
# посмотреть доступные профили
sudo nvidia-smi mig -lgip

# пример: создать инстансы и compute-инстансы
sudo nvidia-smi mig -cgi <profile1>,<profile2> -C
```

```
# проверить, что MIG-устройства появились
nvidia-smi -L
nvidia-smi
```

Важно: пока не созданы GPU instances + compute instances, CUDA-нагрузки не запустятся на MIG, даже если MIG mode включён. [\[48\]](#)

5. Установить Docker (или сразу containerd, если идёшь в Kubernetes).

Дальше поставить NVIDIA Container Toolkit и настроить runtime через nvidia-ctk (официальный способ).

Установка NVIDIA Container Toolkit (apt) и настройка Docker runtime (из официальной инструкции):

```
# репозиторий
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
| sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg

curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.deb \
| sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg]#' \
| sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

sudo apt update
sudo apt -y install nvidia-container-toolkit

# подключить runtime к Docker
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

nvidia-ctk runtime configure правит /etc/docker/daemon.json, после чего требуется рестарт демона Docker.

6. Smoke-test GPU в контейнере:

```
docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
```

7. Smoke-test MIG-пиннинга (запуск контейнера на конкретном MIG UUID): возможность ограничивать видимые MIG-девайсы делается через NVIDIA_VISIBLE_DEVICES/--gpus. [\[48\]](#)

```
# взять MIG UUID из `nvidia-smi -L`
export MIG_UUID="MIG-xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"

docker run --rm --gpus "device=${MIG_UUID}" nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
```

Ветка A: Slurm (батчи/обучение)

В Slurm GPU включаются как GRES: нужно явно задать `GresTypes` и `Gres` в `slurm.conf`, иначе GRES “не существуют” для планировщика. [\[47\]](#)

Для авто-детекта NVIDIA GPU и MIG обычно используют `AutoDetect=nvml` в `gres.conf`, а Slurm затем раздаёт девайсы через `CUDA_VISIBLE_DEVICES` на каждый `step/job`. [\[47\]](#)

1. Установка (минимальный набор, названия пакетов могут отличаться в Ubuntu-репах):

```
sudo apt -y install slurm-wlm munge
sudo systemctl enable --now munge
```

2. Конфиг Slurm (минимально, single-node):

- Включи `select/cons_tres`, потому что `--gpus*` опции поддерживаются только с этим плагином. [\[47\]](#)
- В `slurm.conf` задай GRES ожидания, например 2 GPU или “2 MIG-слота” (в зависимости от того, как ты уже нарезал MIG). [\[47\]](#)

Примерная логика (эскиз, не copy-paste “в лоб”):

```
# /etc/slurm/slurm.conf (эскиз)
GresTypes=gpu
# важно: описать NodeName и Gres=... чтобы slurmctld знал, сколько GPU/GRES ждать
NodeName=server CPUs=<...> RealMemory=<...> Gres=gpu:h100:2
SelectType=select/cons_tres
```

3. `gres.conf` на ноде (авто-детект):

```
# /etc/slurm/gres.conf
AutoDetect=nvml
```

`AutoDetect=nvml` подтягивает конфигурацию GPU из NVML; для MIG Slurm ожидает, что устройства уже нарезаны, и не выполняет динамическое MIG-партиционирование. [\[47\]](#)

4. Проверка и запуск:

```
# посмотреть, как slurmd “видит” железо
sudo slurmd -C
sudo slurmd -G

# пример запуска
srun --gpus=1 --pty bash
echo $CUDA_VISIBLE_DEVICES
nvidia-smi
```

Job’ы не получают GPU, пока явно не запросишь их при сабмите (`--gpus`, `--gres` и т.п.). [\[47\]](#)

Ветка B: Kubernetes + NVIDIA GPU Operator (сервисы/инференс)

GPU Operator разворачивает MIG Manager и умеет управлять MIG-конфигурацией на нодах через label `nvidia.com/mig.config`. [\[49\]](#)

MIG Manager требует, чтобы на GPU, которые переконфигурируются, не работали пользовательские нагрузки; иногда нужен cordon/ребут ноды (особенно в CSP-сценариях). [\[49\]](#)

1. Подготовить containerd под Kubernetes и подключить NVIDIA runtime (официально через `nvidia-ctk`).

```
sudo nvidia-ctk runtime configure --runtime=containerd
sudo systemctl restart containerd
```

2. Поставить Kubernetes (kubeadm/k3s — на твой выбор), затем Helm и репозиторий NVIDIA (опуская стандартные шаги kubeadm, чтобы не раздувать ответ).
3. Установка GPU Operator с включением MIG-стратегии (пример из доков): [\[49\]](#)

```
helm repo add nvidia https://nvidia.github.io/gpu-operator
helm repo update

helm install --wait --generate-name \
  -n gpu-operator --create-namespace \
  nvidia/gpu-operator \
  --version=v25.10.1 \
  --set mig.strategy=single
```

Если драйверы уже установлены на хосте (что обычно так и делается на bare-metal), при установке Operator нужно отключить установку драйвера опцией `--set driver.enabled=false`. [\[49\]](#)

4. Включить/настроить MIG-профиль через label на ноде (пример): [\[49\]](#)

```
kubectl label nodes <node-name> nvidia.com/mig.config=all-1g.10gb --overwrite
kubectl get node <node-name> -o jsonpath='{.metadata.labels}' | jq .
```

По умолчанию Operator создаёт ConfigMap `default-mig-parted-config`, и MIG Manager берёт профили оттуда. [\[49\]](#)

5. Проверка MIG в Kubernetes (через daemonset драйвера): [\[49\]](#)

```
kubectl exec -it -n gpu-operator ds/nvidia-driver-daemonset -- nvidia-smi -L
kubectl get pods -n gpu-operator
```

Какой целевой Ubuntu (22.04 или 24.04) и какой режим планируется в “проде”: больше батч-тренировки (Slurm) или больше online-инференса (Kubernetes)?

теперь дай мануал, как жто все устанавливать

Нижe — пошаговый мануал “поставить всё” на Ubuntu так, чтобы затем можно было выбирать между Slurm и Kubernetes, не ломая GPU/MIG-конфигурацию. Важная идея: **сначала** базовый GPU-стек (driver → MIG → контейнерный runtime), и только потом “ветки” Slurm и Kubernetes. [\[50\]](#)

Предварительные решения

Перед установкой зафиксируй, кто будет управлять MIG-геометрией: либо ты задаёшь MIG статически (и Slurm/K8s только потребляют), либо даёшь Kubernetes GPU Operator динамически менять MIG через MIG Manager (тогда Slurm и любые хостовые GPU-задачи надо останавливать на время reconfigure). [\[51\]](#)

Для H100 (Hopper) режим MIG **не сохраняется** после перезагрузки/выгрузки модулей драйвера, поэтому сразу планируй автоприменение MIG-геометрии при старте (systemd unit + скрипт). [\[50\]](#)

Шаг 1: NVIDIA driver + MIG

1. Базовые пакеты и перезагрузка:

```
sudo apt update
sudo apt -y install build-essential dkms linux-headers-$(uname -r) curl gnupg2 jq
sudo reboot
```

2. Установи NVIDIA datacenter driver (желательно через пакетный менеджер дистрибутива). [\[52\]](#)

Проверка:

```
nvidia-smi
```

3. Включи MIG на каждом GPU:

```
sudo nvidia-smi -i 0 -mig 1
sudo nvidia-smi -i 1 -mig 1
nvidia-smi -i 0 --query-gpu=pci.bus_id,mig.mode.current --format=csv
nvidia-smi -i 1 --query-gpu=pci.bus_id,mig.mode.current --format=csv
```

На Hopper включение MIG не требует GPU reset, но сам факт “MIG Enabled” не гарантирует готовность к запуску CUDA — нужны ещё GI/CI. [\[50\]](#)

4. Создай GPU Instances и Compute Instances (пример; профили подбирай под задачи):

```
# посмотреть доступные профили
sudo nvidia-smi mig -lgip

# пример: нарезать GPU на набор профилей и сразу создать compute instances
# (замени на реальные профили, которые выбрал)
sudo nvidia-smi mig -cgi 1g.10gb,1g.10gb,2g.20gb -C
```

```
# проверить, что MIG-девайсы реально появились
nvidia-smi -L
nvidia-smi
```

Если GI/CI не созданы, CUDA-нагрузки на MIG не поедут, даже если MIG mode включён. [\[50\]](#)

5. Автовосстановление MIG после перезагрузки (минимальный шаблон):

- Создай скрипт, который делает `nvidia-smi -i X -mig 1` и затем `nvidia-smi mig -cgi ... -C`.
- Повесь его на systemd unit `After=multi-user.target` и `After=nvidia-persistenced.service` (если используешь `persistenced`).

Шаг 2: Контейнеры (Docker и/или containerd) + NVIDIA Container Toolkit

NVIDIA Container Toolkit ставится из официального репозитория и затем подключается командой `nvidia-ctk runtime configure`, которая правит конфиги runtime (для Docker — `/etc/docker/daemon.json`, для containerd — drop-in и импорт в config). [\[52\]](#)

Это базовый слой и для “чистого Docker”, и для Kubernetes (через containerd). [\[52\]](#)

1. Установить NVIDIA Container Toolkit (apt, Ubuntu/Debian):

```
sudo apt-get update
sudo apt-get install -y --no-install-recommends curl gnupg2

curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
| sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg

curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit-keyring.gpg \
| sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg]#' \
| sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit
```

2. Если используешь Docker:

```
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

`nvidia-ctk runtime configure --runtime=docker` модифицирует `/etc/docker/daemon.json`, после чего нужен рестарт Docker. [\[52\]](#)

3. Если готовишь Kubernetes (containerd):

```
sudo nvidia-ctk runtime configure --runtime=containerd
sudo systemctl restart containerd
```

Для containerd nvidia-ctk обычно создаёт /etc/containerd/conf.d/99-nvidia.toml и обновляет imports в /etc/containerd/config.toml. [\[52\]](#)

4. Smoke-test GPU в контейнере:

```
docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
```

5. Smoke-test запуска на конкретном MIG-девайсе:

```
# возьми MIG UUID из `nvidia-smi -L`  
export MIG_UUID="MIG-...uuid..."  
  
docker run --rm --gpus "device=${MIG_UUID}" \  
nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi -L
```

Запуск на конкретном MIG-девайсе делается через NVIDIA_VISIBLE_DEVICES или --gpus с указанием MIG-идентификатора. [\[50\]](#)

Шаг 3А: Slurm (очереди/батчи/обучение)

В Slurm GRES по умолчанию “выключены”, поэтому нужно явно прописать GresTypes и Gres в slurm.conf, иначе GPU не будут выделяться задачам.

Для NVIDIA GPU и MIG рекомендуется AutoDetect=nvml в gres.conf; при этом Slurm **ожидает**, что MIG уже нарезан заранее, и не делает динамическое MIG-партиционирование.

1. Поставить Slurm (пакеты зависят от версии Ubuntu/репозитория):

```
sudo apt -y install slurm-wlm munge  
sudo systemctl enable --now munge
```

2. Минимальная идея конфигурации:

- В slurm.conf включи GresTypes=gpu и укажи ожидаемое количество Gres=... на ноде.
- Обязательно используй SelectType=select/cons_tres, иначе опции вида --gpus/--gpus-per-task будут отвергаться.

3. gres.conf (на compute-ноде):

```
# /etc/slurm/gres.conf  
AutoDetect=nvml
```

AutoDetect=nvml умеет детектить NVIDIA GPU (и MIG), но Gres= в slurm.conf всё равно нужен, чтобы контроллер знал, сколько ресурсов ждать.

4. Проверка:

```
sudo slurmd -C  
sudo slurmd -G
```

```
srunc --gpus=1 --pty bash
echo $CUDA_VISIBLE_DEVICES
nvidia-smi
```

Slurm выставляет `CUDA_VISIBLE_DEVICES` для job step'ов, чтобы разным задачам достались разные устройства.

Шаг 3В: Kubernetes + NVIDIA GPU Operator (сервисы/инференс)

GPU Operator разворачивает MIG Manager и умеет управлять MIG-конфигурацией через `label nvidia.com/mig.config`, а для стратегии MIG используется `mig.strategy` (например, `single` или `mixed`). [\[51\]](#)

MIG Manager требует, чтобы во время изменения MIG-геометрии на GPU не работали пользовательские нагрузки, и в некоторых сценариях может потребоваться `reboot/cordon` ноды. [\[51\]](#)

1. Подними кластер (kubeadm/k3s — на выбор) и установи Helm.
2. Установка GPU Operator (пример из документации):

```
helm repo add nvidia https://nvidia.github.io/gpu-operator
helm repo update

helm install --wait --generate-name \
  -n gpu-operator --create-namespace \
  nvidia/gpu-operator \
  --version=v25.10.1 \
  --set mig.strategy=single
```

Если драйвер уже стоит на хосте (что обычно так на bare-metal), при установке укажи `--set driver.enabled=false`. [\[51\]](#)

3. Применить MIG-профиль через `label` ноды:

```
kubectl label nodes <node-name> nvidia.com/mig.config=all-1g.10gb --overwrite
kubectl get node <node-name> -o=jsonpath='{.metadata.labels}' | jq .
```

По умолчанию MIG-конфиги берутся из ConfigMap `default-mig-parted-config` в namespace GPU Operator. [\[51\]](#)

4. Проверка MIG из driver daemonset:

```
kubectl get pods -n gpu-operator
kubectl exec -it -n gpu-operator ds/nvidia-driver-daemonset -- nvidia-smi -L
```

5. Важный баг-угол: в документации отмечено, что на некоторых драйверах серии 570 при смеси MIG-слайсов и “полных GPU” на одной ноде могут возникать проблемы с планированием GPU-подов (Pending), и там приводится рекомендация по downgrade как workaround. [\[51\]](#)

Если нужен максимально "боевой" мануал под твой сценарий, нужны 4 ответа:

1. Ubuntu 22.04 или 24.04?
2. H100 SXM или PCIe (и есть ли NVLink/NVSwitch)?
3. Планируется один пользователь или много (с очередями и квотами)?
4. Что доминирует: обучение (батчи) или инференс (онлайн-сервисы)?

✱

Ubuntu 24.04

h100 sxm(Сервер же),nvlіnk есть

планируется несколько пользователей,но нагрузка по необходимости,так как условно разные части одной команды

прежде всего обучение

раздели еще из разряда: легко поставить-база

трудно установить, техподдержка может не справиться,лучше поставить самостоятельно

Для твоего сценария (Ubuntu 24.04, 2×H100 SXM с NVLink, несколько пользователей, в основном обучение) практичнее ставить "базу" + Slurm как основной прод-контур, а Kubernetes (GPU Operator) держать как экспериментальный/для будущего сервинга. Для удобства ниже разделено на "легко поставить (база)" и "сложно (лучше самому)".

Легко поставить: базовый контур

- NVIDIA-драйвер на Ubuntu Server: ставь через `ubuntu-drivers/APT` (это рекомендуемый путь в документации Ubuntu Server), затем проверка `nvidia-smi`. [\[53\]](#)
- Контейнерный слой (нужен и для Slurm через контейнеры, и для K8s): ставишь NVIDIA Container Toolkit из официального репозитория и подключаешь runtime командой `nvidia-ctk runtime configure` (отдельно под Docker или containerd), после чего перезапускаешь сервис. [\[54\]](#)
- Базовый "multi-user hygiene": отдельные unix-пользователи, SSH по ключам, понятные точки данных (`/data`, `/datasets`, `/checkpoints`), квоты/права (это снижает вероятность "случайно удалили веса/датасет", но не требует редких знаний).

Минимальные команды для toolkit (пример под Docker) прямо соответствуют официальному гайду: `репозиторий → apt install nvidia-container-toolkit → nvidia-ctk runtime configure --runtime=docker → systemctl restart docker`. [\[54\]](#)

Легко поставить: Slurm как основа обучения

Slurm для GPU опирается на GRES: нужно явно включить `GresTypes` и описать ресурсы ноды, иначе задачи не смогут "запрашивать" GPU нормальным способом.

Для современного GPU-расписания Slurm использует `select/cons_tres`, и именно этот плагин добавляет/нормально обрабатывает опции вроде `--gpus`, `--gpus-per-task`, `--cpus-per-gpu`. [\[55\]](#)

Базовый скелет (идея, не полный конфиг):

- `/etc/slurm/slurm.conf`: `GresTypes=gpu, SelectType=select/cons_tres, NodeName=...`
`Gres=gpu:2` (или более детально по типам). [\[55\]](#)
- `/etc/slurm/gres.conf`: `AutoDetect=nvml`, чтобы Slurm подтягивал GPU-устройства через NVML.
- Тест после запуска `slurmd/slurmctld`: `srun --gpus=1 --pty bash` и проверка, что GPU выделен задаче и виден внутри сессии.

Сложно: MIG (и почему это важно заранее)

На H100 одного “включить MIG mode” недостаточно: нужно создать GPU Instances и Compute Instances, иначе MIG-девайсы не появятся как готовые вычислительные устройства. [\[56\]](#)

В документации NVIDIA отмечается, что для Hopper MIG-режим **не сохраняется** после перезагрузки (и вообще после событий, где теряется состояние драйвера), поэтому MIG-геометрию надо автоматически восстанавливать при старте (обычно systemd unit + скрипт с `nvidia-smi mig ...`). [\[56\]](#)

Практичная стратегия под команду и обучение:

- Если вы хотите “много параллельных экспериментов” (несколько моделей одновременно) — режьте MIG заранее на фиксированные профили и используйте их как “слоты” для Slurm. [\[56\]](#)
- Если важнее “максимальная скорость одной большой тренировки” — оставляйте H100 целиком (без MIG) и параллельте по 2 GPU через DDP/NCCL (MIG может мешать, если нужен весь NVLink-домен под одну задачу).

Сложно: Kubernetes + GPU Operator (лучше ставить самому)

GPU Operator умеет управлять MIG через MIG Manager и применять MIG-конфигурации на ноду через `label nvidia.com/mig.config`, что удобно, но добавляет сложность и риск конфликтов с не-K8s нагрузками. [\[57\]](#)

В типовой установке GPU Operator через Helm можно сразу задать `mig.strategy` (например, `single`), а затем назначать MIG-профиль на ноду через `label`, чтобы MIG Manager применил разметку. [\[57\]](#)

Почему “сложно” и обычно не для техподдержки:

- Нужно уверенно администрировать containerd/CRI, сетевую подсистему K8s, политики, а также понимать, что MIG-reconfigure требует, чтобы на GPU не было активных задач (иначе будут сбои/простой). [\[57\]](#)
- Для HPC-обучения в небольшой команде Kubernetes часто оказывается “дороже” по сопровождению, чем Slurm, если основная цель — очереди и честное деление GPU между пользователями.

Если нужно, можно дать конкретный “боевой” шаблон под ваш режим: **Slurm + (Docker или Apptainer) + фиксированный MIG**, включая systemd-юнит для восстановления MIG,

пример `slurm.conf/gres.conf`, и набор готовых sbatch-шаблонов под PyTorch DDP.



1. <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/>
2. <https://slurm.schedmd.com/gres.html>
3. <https://github.com/NVIDIA/nccl/blob/master/README.md>
4. <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>
5. <https://github.com/NVIDIA/nvidia-container-toolkit/blob/main/README.md>
6. <https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/gpu-operator-mig.html>
7. <https://docs.pytorch.org/docs/stable/generated/torch.nn.parallel.DistributedDataParallel.html>
8. <https://docs.nvidia.com/datacenter/dcgm/latest/gpu-telemetry/dcgm-exporter.html>
9. <https://github.com/NVIDIA/dcgm-exporter>
10. https://docs.nvidia.com/datacenter/tesla/pdf/MIG_User_Guide.pdf
11. <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/getting-started-with-mig.html>
12. <https://www.scaleway.com/en/docs/gpu/how-to/use-nvidia-mig-technology/>
13. <https://massedcompute.com/faq-answers/?question=How+do+I+configure+MPS+to+work+with+MIG+on+the+H100+PCIe%3F>
14. <https://github.com/dholt/slurm-gpu>
15. <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/supported-mig-profiles.html>
16. <https://dev.to/murajo/visualizing-gpu-metrics-with-dcgm-exporter-1268>
17. <https://slurm.schedmd.com/gres.conf.html>
18. <https://www.advancedclustering.com/wp-content/uploads/2022/03/gtc22-whitepaper-hopper.pdf>
19. <https://ayatk.hatenablog.com/entry/2018/10/31/200612>
20. <https://github.com/NVIDIA/nvidia-container-toolkit>
21. <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/arch-overview.html>
22. <https://nvidia.github.io/libnvidia-container/>
23. <https://best-of-web.builder.io/library/NVIDIA/nvidia-container-toolkit>
24. <https://docs.nvidia.com/deeplearning/nccl/index.html>
25. <https://glaringlee.github.io/generated/torch.nn.parallel.DistributedDataParallel.html>
26. <https://unrealcontainers.com/docs/concepts/nvidia-docker>
27. <https://developer.nvidia.com/nccl>
28. https://sagemaker.readthedocs.io/en/v2.76.0/api/training/sdp_versions/v1.0.0/smd_data_parallel_pytorch.html
29. <https://sarus.readthedocs.io/en/1.3.3/config/nvidia-container-toolkit.html>
30. https://github.com/NVIDIA/nccl/tree/inc_nsteps
31. <https://slurm.schedmd.com/gres.html>
32. <https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/gpu-operator-mig.html>
33. <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/>
34. <https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/1.9.0/gpu-operator-mig.html>

35. <https://github.com/NVIDIA/k8s-device-plugin>
36. <https://github.com/NVIDIA/k8s-device-plugin/issues/767>
37. <https://github.com/dholt/slurm-gpu>
38. https://www.escape-technology.de/images/news/2025/NVIDIA_MIG/MIG_User_Guide.pdf
39. <https://docs.nvidia.com/datacenter/cloud-native/kubernetes/latest/index.html>
40. <https://slurm.schedmd.com/gres.conf.html>
41. <https://mgujob.tistory.com/20>
42. <https://developers.redhat.com/articles/2025/05/27/boost-gpu-efficiency-kubernetes-nvidia-mig>
43. <https://github.com/NVIDIA/gpu-operator/issues/981>
44. https://github.com/NVIDIA/gpu-operator/blob/main/assets/state-mig-manager/0400_configmap.yaml
45. <https://github.com/NVIDIA/gpu-operator/issues/603>
46. <https://www.scaleway.com/en/docs/gpu/how-to/use-nvidia-mig-technology/>
47. <https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/gpu-operator-mig.html>
48. <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/>
49. https://docs.nvidia.com/datacenter/tesla/pdf/MIG_User_Guide.pdf
50. <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/>
51. <https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/gpu-operator-mig.html>
52. https://docs.nvidia.com/datacenter/tesla/pdf/MIG_User_Guide.pdf
53. <https://documentation.ubuntu.com/server/how-to/graphics/install-nvidia-drivers/>
54. https://docs.nvidia.com/datacenter/tesla/pdf/MIG_User_Guide.pdf
55. https://slurm.schedmd.com/cons_tres.html
56. <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/>
57. <https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/gpu-operator-mig.html>
58. <https://cloudspinx.com/how-to-install-and-use-apptainer-on-ubuntu-debian/>
59. <https://stackoverflow.com/questions/66092965/correct-usage-of-gpus-per-task-for-allocation-of-distinct-gpus-via-slurm>
60. <https://www.youtube.com/watch?v=Se5THDk0c5Q>
61. <https://www.youtube.com/watch?v=ZACVgNrc-ek>
62. https://slurm.schedmd.com/SLUG19/GPU_Scheduling_and_Cons_Tres.pdf
63. <https://www.youtube.com/watch?v=Z29Fpx8KClw>
64. <https://apptainer.org/docs/admin/main/installation.html>
65. https://hostkey.com/documentation/technical/gpu/nvidia_gpu_linux/
66. <https://github.com/apptainer/apptainer/blob/main/INSTALL.md>
67. <https://linuxconfig.org/how-to-install-nvidia-drivers-on-ubuntu-24-04>
68. <https://docs.nvidia.com/datacenter/tesla/driver-installation-guide/index.html>
69. <https://github.com/oddmario/NVIDIA-Ubuntu-Driver-Guide>
70. <https://www.cherryserver.com/blog/install-nvidia-drivers-ubuntu>

